



Decision Theory

Daniel C



Roadmap

From classical choice to embedded counterfactuals

- **Why decision theory matters for AI safety** (modelling, specification, reflective stability, multi-agent conflict)
- **Why it is hard:** optimisation is solved for *dualistic* agents but not for *embedded* ones, where the agent must *construct* counterfactuals
- **The candidates:** causal (CDT), evidential (EDT), functional/logical (FDT), updateless (UDT 1.0, UDT 1.1) – each is a different recipe for the counterfactual
- **The hard core:** problems with UDT, the 5-and-10 problem, logical counterfactuals and Löb's theorem
- **A unifying goal:** formalise decision problems as programs and ask which decision theory is *optimal* across all “fair” problems



Why study decision theory? (for AI safety)

Four reasons

- **Architecture-independent modelling.** We want to reason about how a superintelligent agent will *behave* without knowing the details of how it is built. Decision theory is a minimal model of “what is rational choice”, capturing behaviour any sufficiently capable agent tends to converge on.
- **Implementation / the specification–optimisation split.** Alignment decomposes into (1) *specify* a goal we want, and (2) *optimise* it correctly. Decision theory is part (2). A perfect specification is **not sufficient** for safety: if the agent constructs the wrong counterfactuals, it can “optimise” the goal in a way we never intended.
- **Reflective stability.** Safety properties must remain invariant under self-modification (cf. Agent Foundations deck). Decision theory is the simplest setting to ask: *would an agent running decision theory A want to self-modify into, or build a successor with, decision theory B?* A decision theory that wants to overwrite itself is a warning sign.
- **Multi-agent dynamics.** Decision problems get hard under strategic interaction and mutual modelling. We want a decision theory under which agents can *cooperate* with other superintelligences and *avoid catastrophic conflict*, without being *exploitable*.



The easy case: dualistic agents

When the agent is cleanly separated from the world

- A *dualistic* agent has clean input/output channels with its environment (the “cybernetic” picture: a player optimising a video game). It is “**larger than**” the world (can hold a full model of it) and “**outside**” it (need not reason about itself).
- Here the action is a genuine *free variable*: nothing in the world depends on the agent’s internal computation except through the downstream consequences of the action. Choosing optimally is conceptually solved:

$$a^* = \arg \max_a \mathbb{E}[U \mid \text{do}(A = a)].$$

- This is the mature, successful framework of **expected-utility maximisation**: the von Neumann–Morgenstern / Savage axioms (preferences satisfying coherence are EU-maximisation), Bayesian updating, and **complete-class theorems** (every admissible decision rule is Bayes-optimal for some prior).
- It underpins modern economics, statistics, and reinforcement learning. The conceptual work is done; only computation remains.



The hard case: embedded agents

When the agent is a part of the world it reasons about

- An *embedded* agent is a piece of the world it is trying to optimise (cf. Agent Foundations deck). It can be copied, predicted, simulated; its source code can be read by others.
- Now **“which action I take” is just another fact about the world**, fixed by physics (or by the agent's own deterministic computation). There is no free variable to set, and **no ready-made answer to “what would happen if I acted differently”** – the alternative may be physically or logically impossible.
- So the counterfactual must be *constructed*, not read off. **This is the whole difficulty**: decision theory for embedded agents is the problem of constructing counterfactuals.
- Each decision theory is a different construction:
 - **EDT**: condition on the action as *evidence*, $\mathbb{E}[U \mid A = a]$
 - **CDT**: *intervene* causally on the action, $\mathbb{E}[U \mid \text{do}(A = a)]$
 - **FDT/UDT**: reason about the *logical output* of the decision procedure being a



A 60-second primer: Bayesian networks

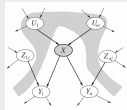
Encoding “what depends on what”

- A Bayesian network is a directed acyclic graph whose nodes are random variables and whose edges mean “directly listens to”. The joint distribution *factorises* along the graph:

$$P(X_1, \dots, X_n) = \prod_{i=1}^n P(X_i \mid \text{Pa}(X_i)),$$

where $\text{Pa}(X_i)$ are the parents (direct causes) of X_i .

- Graphically: each variable is independent of its non-descendants given its parents. The graph is a compact, structured stand-in for a huge joint distribution.





Causal decision theory: seeing vs. doing

The do-operator

- Pearl's key distinction: *observing* $X=x$ (passive evidence) is not the same as *doing* $X=x$ (an intervention).
- $\text{do}(X = x)$ models an intervention: **sever all arrows into X** (cut X off from its causes), fix $X = x$, then propagate downstream. Writing $\text{Pa}(X)$ for the direct causes of X :

$$P(Y \mid \text{do}(X=x)) = \sum_z P(Y \mid X=x, \text{Pa}(X)=z) P(\text{Pa}(X)=z).$$

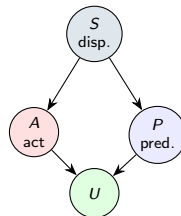
- Classic example: a barometer reading is correlated with storms (common cause: air pressure). *Seeing* a low reading predicts a storm; *forcing* the needle down (an intervention that severs it from air pressure) tells you nothing about storms.
- **CDT**: choose $a^* = \arg \max_a \mathbb{E}[U \mid \text{do}(A = a)]$. Evaluate each action by the causal consequences of *intervening* on it, holding the rest of the past fixed.



CDT on Newcomb's problem

Two-boxing, and why

- **Newcomb:** a transparent box holds \$1,000; an opaque box holds \$1,000,000 *iff* a highly reliable predictor Ω foresaw you would take *only* the opaque box. Ω has already chosen.
- CDT applies $\text{do}(A)$: this **severs** your action from your disposition S , so it no longer carries information about Ω 's prediction P . The box contents are now fixed; taking both boxes earns \$1,000 more *in every fixed state*. CDT **two-boxes**.
- But predictably: two-boxers are predicted to two-box and walk away with \$1,000; one-boxers get \$1,000,000.
- CDT wins the battle (local dominance) and loses the war (the predictable policy is poor). It *deliberately discards* the dependence of Ω 's prediction on the agent's decision.



CDT cuts $S \rightarrow P$, losing the link to P .



CDT vs. physics: the intervention is a fiction

Does CDT really “reflect physical reality”? (following Ebtekar & Hutter)

- CDT's $\text{do}(A = a)$ holds the past fixed, **magically sets the action**, and propagates forward. What licenses cutting the action loose from its causes?
- In a *deterministic, time-reversible* physical world, your action has physical antecedents (your brain state, your source code). A predictor that reads those antecedents is correlated with your action through **perfectly ordinary physics**, not magic.
- “Severing the incoming arrows to *A*” directly **contradicts embeddedness**: it pretends your action has no causes, but physically it does. The intervention is a harmless idealisation for a dualistic agent; for an embedded agent it *throws away real correlations* (with predictors, copies, simulations).
- So the usual defence is backwards: physics gives no privileged metaphysical “intervention” that frees the action from its past. This pressures us to move the counterfactual off the *physical act* and onto the agent's *policy / computation*.



Evidential decision theory: condition, don't intervene

EDT as conditioning

- EDT evaluates an action by the *news* it would bring – it **conditions** on “I do a ” as evidence:

$$a^* = \arg \max_a \mathbb{E}[U \mid A = a].$$

- On **Newcomb**: one-boxing is strong evidence the opaque box is full (the predictor is reliable), so $\mathbb{E}[U \mid A = \text{one-box}]$ is huge. EDT **one-boxes** and gets the million. EDT captures exactly the predictive dependence CDT discards.
- The contrast in one line: $\mathbb{E}[U \mid \text{do}(A=a)]$ (CDT, severs the action's causes) vs. $\mathbb{E}[U \mid A=a]$ (EDT, keeps them as evidence).



EDT mistakes evidence for control

The smoking lesion

- A genetic lesion L causes cancer (cost C) and makes you more likely to want to smoke. Smoking itself is harmless and gives benefit $b > 0$.
- Conditioning on "I smoke" raises $P(L)$, hence raises estimated cancer risk: $\mathbb{E}[U \mid \text{smoke}] = b - C P(L \mid \text{smoke})$. For strong correlation, EDT **abstains** – even though abstaining does *nothing* to reduce cancer. EDT confuses a **symptom** for a **lever**.
- **Scorecard:** CDT and EDT are each half right.
 - CDT ignores predictive dependence $\Rightarrow$ loses Newcomb.
 - EDT overreacts to mere correlation $\Rightarrow$ loses the smoking lesion.
- The missing ingredient is a dependence that is neither raw physical causation nor raw evidential correlation.



Spurious counterfactuals: EDT breaks for self-aware agents

Conditioning on a probability-zero action

- A capable embedded agent can reason about (even *prove* things about) its own source code, so it can largely predict its own action. But then, for the action a it will *not* take, $P(A = a) = 0$.
- EDT's recipe $\mathbb{E}[U \mid A = a]$ is then $\frac{0}{0}$ – **undefined**. You may assign the unchosen action *any* value you like: a **spurious counterfactual**.
- So conditioning gives *no constraint* on counterfactual actions you are certain you won't take – exactly the cases decision-making turns on.
- (Foreshadow: in proof-theoretic form this same pathology becomes the **5-and-10 problem**, and the fix needs a notion of *logical* dependence.)



Functional decision theory: choose the algorithm's output

Identify with the computation, not the body

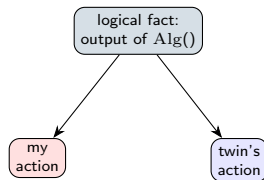
- The shift: you are not just *this physical body about to move*; you are an **implementation of a decision computation**. Predictors, copies, and simulations elsewhere in the world may be other implementations or correlates of the *same* computation.
- FDT's counterfactual: "consider the world in which this decision algorithm, on this input, outputs *a*." Everything **logically downstream** of that output changes with it.
- **Newcomb**: "my algorithm outputs one-box" $\Rightarrow \Omega$ (who computed my algorithm) predicted one-box $\Rightarrow$ box full. FDT one-boxes, *for the right reason*. **Smoking lesion**: "my algorithm outputs smoke" does not change the lesion (the lesion isn't running my decision), so FDT smokes. **FDT gets both right**.
- Slogan: *CDT asks what follows from the act being physically different; EDT, from the act being evidence; FDT, from the decision computation being different.*



Twin Prisoner's Dilemma & subjunctive dependence

Same function, two bodies

- You and a psychological *twin* (your exact copy) play a one-shot Prisoner's Dilemma in separate rooms, no communication. You run the *same* decision function.
- **CDT**: "I can't causally affect her; defection dominates." Both defect, each gets \$1.
- **FDT**: "We compute the same function. If it outputs *C*, both cooperate; if *D*, both defect." So FDT cooperates – both get \$3.
- **Subjunctive dependence**: two physical systems both depend on one *logical* fact (the function's output). FDT acts as if it controls *all* instances of the computation at once.



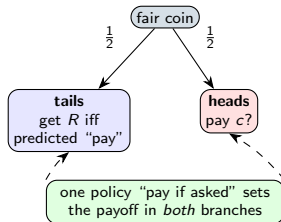
one logical cause, two physical effects: CDT sees no arrow between them, FDT sees the shared parent



Counterfactual mugging: the updateless move

Paying in a branch you know you're not rewarded in

- Ω flips a fair coin. **Tails:** it gives you $R = \$10,000$ iff it predicted you would pay on heads. **Heads:** it asks you for $c = \$100$. You wake up, you are asked to pay (so: heads). Pay?
- **Updateful** reasoning conditions on "I'm in heads": it sees only the \$100 cost and **refuses**. But *ex ante* the policy "pay if asked" is worth $\frac{1}{2}(R - c) > 0$.
- **UDT:** do not condition away the (tails) branch whose payoff *depended on your policy*. Evaluate the output against the **prior**. UDT **pays**.
- Updatelessness is *not* ignorance: you still use the observation; you just refuse to delete branches whose payoffs hinge on your policy. With no predictor, UDT = ordinary EU maximisation.
- **Reflective stability:** an updateful agent here *wishes it had precommitted* to pay. UDT already follows the policy it would have committed to – nothing to self-modify.





UDT 1.1: optimise the whole policy, not one action

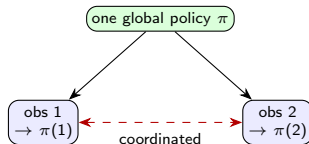
When local outputs must be coordinated

- Even *updateless local-action* selection can be too local. **Copied-agent anti-coordination**: two copies see index 1 or 2, must pick *different* actions to win (10 each), else 0.
- The local question “if $\text{Alg}(1) = A$, what is $\text{Alg}(2)$?” is the *wrong type*: both symmetric answers fail. The good object is the *whole table* $\text{Alg} : \{1, 2\} \rightarrow \{A, B\}$.

- **UDT 1.1 (Wei Dai)**: first choose the entire policy $\pi^* = \arg \max_{\pi} \mathbb{E}_{\text{prior}}[U]$, then apply π^* to your actual observation:

$$\pi^* \in \arg \max_{\pi \in \Pi} \mathbb{E}_{K^{\text{Pol}}_{\pi}}[U], \quad \text{UDT1.1}(x) = \pi^*(x).$$

- π_{AB}, π_{BA} score 10; a shared tie-breaker coordinates the copies. This is **global optimisation over branches**, not local optimisation in one branch.





The three shifts in “what am I choosing?”

CDT/EDT → FDT → UDT 1.0 → UDT 1.1 (after Kosoy's formalism)

Rule	Object chosen	Update status	Dependence used
CDT	local physical act	updateful	causal intervention
EDT	local act (as evidence)	updateful	evidential correlation
FDT / TDT	local algorithm output	updateful	logical / subjunctive
UDT 1.0	local algorithm output	updateless	logical / subjunctive
UDT 1.1	whole policy	updateless	logical over policies

- **Shift 1 (FDT):** from a physical act to the output of an abstract computation – captures predictors, copies, simulations.
- **Shift 2 (UDT 1.0):** from posterior to prior – keep the value of being predictable, don't discard branches whose payoff your policy controls.
- **Shift 3 (UDT 1.1):** from a single output to a whole policy – coordinate behaviour across all possible observations.



Problem 1: paying for worlds that don't exist

Ex-ante optimality is only as good as the prior

- UDT maximises *prior* expected utility. In the branch it actually inhabits it will **knowingly burn real utility**. In counterfactual mugging, after seeing heads, UDT hands over a real \$100 in the only world it will ever experience, to benefit a tails-world *it now knows did not happen*.
- Generalise: a UDT agent will take actions that are bad *in the actual world* because they would have been good in counterfactual scenarios that are never realised.
- This is dangerous when the prior is even slightly wrong about *which worlds are possible*: put prior mass on a predictor that does not actually exist (or on a logically impossible scenario), and UDT pays a **real, ongoing tax** for fake benefits. A clever (or merely hypothetical) Ω can extract resources by promising/threatening in branches that never occur.
- The worry: is “best from the prior” the right notion of rational, when “the prior” ranges over worlds you have since learned are not actual?



Problem 2: logical updatelessness

Which prior, when some of your uncertainty is mathematical?

- A real embedded agent's uncertainty is not only *empirical* (coin flips). It is also **logical**: it does not know all consequences of its own source code, the 10^{100} -th digit of π , or whether the opponent's program halts.
- UDT says "don't update." Refusing to update on *empirical* observations is clear. Refusing to update on *logical* facts is not: as the agent thinks longer it *learns* logical facts (proves theorems, computes digits) – and that looks exactly like updating.
- **Logical updatelessness** would have the agent choose its policy as if from a state of *logical* ignorance, not taking computed logical facts into account. But (a) it cannot help computing some of them, and (b) we have **no coherent "logical prior"** – a probability distribution over mathematical truths before any computation.
- This is open. A satisfactory UDT needs a theory of *logical uncertainty* (cf. logical induction), which we do not yet have. It is also the doorway to the 5-and-10 problem.



The 5-and-10 problem

The simplest decision, made impossible by self-knowledge

- You may take \$5 or \$10; utility = the dollars. Obviously take \$10. Can a fully self-aware agent?
- An embedded agent has its *own source code* and can prove things about it, hence prove facts about **its own action**. Its action is just another theorem about the world.
- **Naive proof-based agent:** search for proofs of statements " $(A=\$5) \Rightarrow U=x$ " and " $(A=\$10) \Rightarrow U=y$ ", then take the action with the higher *proven* payoff.
- **The trap.** The agent is deterministic: it takes exactly one action. For the action it does *not* take – say \$10 – the conditional " $(A=\$10) \Rightarrow (U=\$0)$ " is **vacuously true** (false antecedent). If the agent can prove it does not take \$10, it can prove *anything* about that branch – a **spurious counterfactual** – conclude "\$10 yields \$0", and take \$5.



A self-contained look at Löb's theorem

When “provable” is allowed to behave like “true”

- Fix a formal system (e.g. PA). Write $\Box P$ for the *sentence* “ P is provable”. Keep two things distinct: $\Box P$ is a statement *inside* the system; “the system proves P ” (written $\vdash P$) is a fact *about* the system.
- Naively one expects the reflection schema $\Box P \rightarrow P$ (“if it’s provable, it’s true”). **Löb’s theorem** says the system can prove $\Box P \rightarrow P$ *only for* P it can already prove:

if $\vdash (\Box P \rightarrow P)$ then $\vdash P$; equivalently $\vdash \Box(\Box P \rightarrow P) \rightarrow \Box P$.

- Intuition (Curry-style): the self-referential sentence “*if this sentence is provable, then* P ” forces $\vdash P$ whenever the system trusts its own proofs about P .
- Special case $P = \perp$: a consistent system **cannot prove its own consistency** ($\Box \perp \rightarrow \perp$), recovering Gödel’s second incompleteness theorem.



Why Löb makes the spurious proof available

The self-fulfilling proof

- Let $S := ((A=\$10) \Rightarrow (U=\$0))$. The agent is built so that *proving* S leads it to take \$5.
- Then $\Box S \rightarrow S$ is provable: if it is provable that \$10 yields \$0, the agent takes \$5, so $A \neq \$10$, so S is vacuously true. By **Löb's theorem**, $\vdash S$. The agent proves the spurious counterfactual and takes \$5, getting \$5.
- So Löb shows the bad statement *is genuinely provable* – a naive proof search is **not safe** from being driven to the worse action. Ruling out spurious proofs needs extra machinery (e.g. “playing chicken with the universe”), and a general theory of *logical counterfactuals* (counterpossibles) remains **open**.
- **Embedded-agency moral:** the agent's action is just another fact it can prove about the world; its self-knowledge *undermines the very counterfactuals* decision-making needs. (The same Löbian obstacle blocks an agent from fully trusting a successor's proofs – a reflective-stability problem.)



Why we want a formal optimality criterion

Beyond case-by-case scoring

- Newcomb, counterfactual mugging, the 5-and-10 problem, the twin PD, open-source games all share a **common structure**: agent and environment are both *programs* that may inspect each other's code, simulate each other's behaviour, or contain subroutines algorithmically *isomorphic* to one another.
- Yet these are usually presented as standalone puzzles, and CDT/EDT/FDT/UDT are judged *case by case* ("does it get Newcomb right?") rather than against a single optimality standard.
- **The substantive question**: can we define one class of computational environments rich enough to contain all these problems at once, together with a criterion of optimality that any candidate decision theory can be *measured* against?



Agents and worlds as programs

The setup

- **Agent** = a program with access to (i) its own source code and (ii) the world's source code; it returns an action.
- **World** = a program that may
 - *call the agent* as a function (input $\rightarrow$ action), possibly *many times on different inputs* ("what would the agent do if it observed x ?"), and
 - *search for proofs* about the agent's input-output behaviour;it returns a utility.
- **Goal:** design an agent whose utility is high across (almost) all worlds – the best mapping from situations to actions.





Fairness

Judging the agent only on its behaviour

- Without a constraint, a “world” could just punish source code: “if the agent is written in Python, utility = 0.” Then *no* decision theory could do well and the criterion would be vacuous.
- **Fairness:** the world may depend on the agent *only through its input–output behaviour* (its policy), never through implementation details unrelated to its outputs. Equivalently, a fair world is representable as a program that takes the agent's *policy/behaviour* as input.
- Newcomb, the smoking lesion, counterfactual mugging, the twin PD, and the open-source PD are all **fair**: the predictor/lesion/copy interacts with the agent through its behaviour. This is what makes them legitimate tests.



What a good agent must discover

Two demands – which are exactly FDT/UDT turned into a spec

- **Coordinate across function calls.** If the world queries the agent on several inputs (probing its counterfactual behaviour), the agent must *recognise this* and keep its counterfactual outputs *coordinated* for high utility. (This is precisely UDT 1.1's move to optimise a whole policy.)
- **Subjunctive dependence.** If the world contains a subroutine *algorithmically isomorphic* to the agent (a copy in different code), the agent should treat its own decision as also fixing that subroutine's output – and so must *detect* that “the world contains something isomorphic to me.” (This is precisely FDT.)

The open problem (and the bridge ahead)

- Define the class of fair computational worlds precisely; define optimality (e.g. dominance across all fair worlds); show CDT/EDT/FDT/UDT differ on it; and ask whether any (or some modification) is *optimal*.
- Caveats already met: logical counterfactuals (5-and-10), logical uncertainty / logical updatelessness, what counts as a “policy”, and multi-agent fairness & bargaining – which lead into reflective oracles, open-source games, and commitment races.